



Universidad de Concepción

Facultad de Ingeniería

Departamento de Ing. Informática y Ciencias de la Computación

Ingeniería Civil Informática

Estructuras de Datos

Análisis Comparativo de Tablas Hash para el Procesamiento de Volúmenes de Datos

Tarea 3

Integrantes:

Nicolás Ricciardi

Enzo Levancini

Máximo Beltrán

Julio 2026

Índice

1. Introducción	2
2. Descripción de las Estructuras de Datos Comparadas	2
2.1. Funciones Hash	2
2.2. Tabla Hash con Hashing Abierto (encadenamiento)	3
2.3. Tabla Hash con Hashing Cerrado (direccionamiento abierto)	3
2.4. <code>std::unordered_map</code> (STL de C++)	3
3. Descripción del Dataset y del Ambiente de Experimentación	3
3.1. Dataset: Tweets de las Elecciones Australianas 2019	3
3.2. Lectura del Dataset	4
3.3. Ambiente de Hardware y Software	4
4. Resultados Experimentales	4
4.1. Tamaño de las Estructuras de Datos en Memoria	4
4.2. Rendimiento en la Creación y Escalabilidad	5
4.3. Análisis del Impacto del Tipo de Clave	6
5. Conclusiones	6

Introducción

Una **tabla hash** es una estructura de datos que implementa un diccionario: asocia claves con valores y ofrece inserción y búsqueda en tiempo esperado $O(1)$, apoyándose en una **función hash** que transforma cada clave en un índice de un arreglo. Su rendimiento real depende de decisiones de diseño concretas: cómo se resuelven las **colisiones** (dos claves que caen en el mismo índice), cuánto se permite llenar la tabla antes de agrandarla, y qué tan costoso es hashear y comparar el tipo de clave utilizado.

En este trabajo se comparan experimentalmente esas decisiones sobre un problema concreto de procesamiento de volúmenes de datos: **contar cuántos tweets publicó cada usuario** en un dataset de 183.370 tweets sobre las elecciones australianas de 2019. Por cada tweet se consulta la tabla con la clave del usuario y se incrementa su contador, insertando la clave si no existe, lo que somete a la estructura a una carga intensiva de búsquedas e inserciones.

Se evalúan tres estructuras: una tabla hash con **hashing abierto** (encadenamiento) y otra con **hashing cerrado** (direccionamiento abierto), ambas de implementación propia — la segunda con sus tres estrategias de sondeo: *linear probing*, *quadratic probing* y *double hashing* —, más `std::unordered_map` de la STL de C++ como referencia. Cada estructura se prueba con dos claves que identifican unívocamente al usuario: `user_id` (entero de 64 bits) y `user_screen_name` (cadena de texto), lo que exige funciones hash para ambos tipos.

Las implementaciones se desarrollaron en C++17 (MinGW-w64 GCC 14.2.0); los experimentos se orquestaron con un script Bash que exporta los resultados a archivos CSV, y el análisis estadístico y los gráficos se generaron con Python (pandas y matplotlib). Como adelanto de las conclusiones: el hashing cerrado con quadratic probing resultó la variante más rápida, la clave numérica `user_id` superó consistentemente a `user_screen_name` en tiempo y memoria, y el hashing abierto fue el más económico en memoria a costa de ser el más lento.

Descripción de las Estructuras de Datos Comparadas

Los códigos fuentes documentados de las implementaciones analizadas se encuentran adjuntos y en el siguiente repositorio de acceso público: <https://github.com/soda-caustica/estructuras-3>. El desarrollo sigue el estándar de codificación adoptado para el curso.

Funciones Hash

Cada tipo de clave requiere su propia función de dispersión, y el double hashing exige además una segunda función por tipo. Para `user_id` (entero de 64 bits) se usa como hash principal **SplitMix64** [4], un mezclador multiplicativo de bits que difunde IDs consecutivos uniformemente por la tabla, y como hash secundario **component sum** [3], que suma los 8 bytes del entero. Para `user_screen_name` (string) el hash principal es la **acumulación polinomial** con base 31 evaluada mediante la regla de Horner [3], y el secundario es **FNV-1a** de 64 bits, un hash que combina XOR y multiplicación por un primo [3]. Todas retornan un valor sin signo que se reduce módulo la capacidad

(prima) de la tabla.

Tabla Hash con Hashing Abierto (encadenamiento)

Implementación propia donde cada celda del arreglo es una lista enlazada con todos los pares (clave, contador) cuyo hash cae en esa celda, de modo que la tabla nunca se llena. Inserción y búsqueda cuestan $O(1 + \lambda)$ esperado y $O(n)$ en el peor caso, con $\lambda = n/N$ el factor de carga [1]. Parámetros: capacidad inicial de 11 buckets, capacidad siempre prima, factor de carga máximo $\lambda = 1,0$; al superarlo se duplica la capacidad al siguiente primo y se redistribuyen las claves (rehash, $O(n)$ amortizado).

Tabla Hash con Hashing Cerrado (direccionamiento abierto)

Implementación propia donde todos los pares viven dentro del mismo arreglo de celdas, lo que favorece la localidad de caché. Ante una colisión se sondean otras celdas según la estrategia: *linear probing* ($h_1 + j$), *quadratic probing* ($h_1 + j^2$) o *double hashing* ($h_1 + j \cdot p$, con $p = q - (h_2 \bmod q)$ y q el mayor primo menor que N). Inserción y búsqueda cuestan $O(1)$ esperado mientras $\lambda \leq 0,5$ y $O(n)$ en el peor caso. Parámetros: capacidad inicial 11, capacidad siempre prima y factor de carga máximo $\lambda = 0,5$, condición que garantiza que el sondeo cuadrático siempre encuentra una celda libre [2].

`std::unordered_map` (STL de C++)

Contenedor estándar de la STL usado como línea base, envuelto en una clase con la misma interfaz que las implementaciones propias para que el código experimental trate a las tres estructuras de manera uniforme. Implementa hashing abierto con encadenamiento por buckets; inserción y búsqueda cuestan $O(1)$ promedio y $O(n)$ en el peor caso. Se utilizó con su configuración por defecto (función hash `std::hash`, factor de carga máximo 1,0), precisamente porque su rol es servir de referencia sin ajustes.

Descripción del Dataset y del Ambiente de Experimentación

Dataset: Tweets de las Elecciones Australianas 2019

Característica	Valor
Filas de datos	183.370 tweets
Usuarios únicos	45.862
Columnas	11 (se usan <code>user_id</code> y <code>user_screen_name</code>)
Tamaño en disco	70,8 MB
Período	10 al 20 de mayo de 2019
Formato	CSV con campos entre comillas (RFC 4180)

Lectura del Dataset

No fue necesario modificar ni limpiar el dataset: no presenta valores vacíos en las claves utilizadas y todos los `user_id` son enteros de 64 bits válidos. Sí requirió un lector de CSV propio conforme a RFC 4180, porque campos como `full_text` contienen comas, comillas escapadas y **saltos de línea internos**: el archivo tiene 377.704 líneas físicas pero solo 183.370 filas reales, por lo que un parseo ingenuo línea a línea corrompería los datos. Para eficiencia, el archivo completo se carga a memoria de una vez y se parsea desde ahí; de cada fila se conservan solo las dos claves de interés (`user_id` como `uint64_t` y `user_screen_name` como `std::string`), ubicadas por nombre de columna en el encabezado. La carga y el parseo ocurren *antes* de iniciar cada medición, de modo que los tiempos reportados corresponden exclusivamente al trabajo de las tablas hash.

Ambiente de Hardware y Software

Componente	Detalle
Procesador	Intel Core i5-9300H @ 2,40 GHz (4 núcleos / 8 hilos)
Memoria RAM	24 GB DDR4 (16 GB @ 3200 MHz + 8 GB @ 2667 MHz)
Memoria Caché	L1: 256 KB, L2: 1 MB, L3: 8 MB
Sistema Operativo	Windows 11 Home, 64 bits
Compilador	MinGW-w64 GCC 14.2.0, banderas <code>-std=c++17 -O2</code>

Cada medición se ejecuta como un **proceso independiente** lanzado por un script Bash, que imprime una línea CSV por experimento (separada por “;” según el formato solicitado) y acumula los resultados en `results/`. Cada experimento se repitió **20 veces**, reportándose promedio y desviación estándar. El tiempo se mide con `std::chrono::steady_clock`, un reloj monótono inmune a ajustes del reloj del sistema.

Resultados Experimentales

El conteo identificó **45.862 usuarios únicos**, resultado idéntico con ambas claves y en las tres estructuras, lo que valida la corrección de las implementaciones. Cada medición registra, en una única pasada sobre el dataset, el tiempo acumulado al alcanzar cada punto de control (10.000, 20.000, ..., 180.000 tweets); el proceso se repitió 20 veces por combinación de estructura y clave. La memoria se mide en una pasada separada de las cronometradas, ya que su valor es determinista (mismo orden de inserción) y recorrer la tabla para medirla sesgaría los tiempos.

Tamaño de las Estructuras de Datos en Memoria

La siguiente tabla resume el consumo con el dataset completo (180.000 tweets procesados). Las tres variantes de hashing cerrado consumen **exactamente la misma memoria** en todos los puntos de control: la estrategia de sondeo solo determina cómo se recorren las celdas ante una colisión, no cuántas se reservan, y las tres comparten idéntica política de crecimiento; por ello se reportan como una sola fila.

Estructura	Con user_id	Con user_screen_name
Hashing abierto	1,42 MB	2,45 MB
Hashing cerrado (3 variantes)	1,57 MB	3,92 MB
<code>std::unordered_map</code>	1,68 MB	2,71 MB

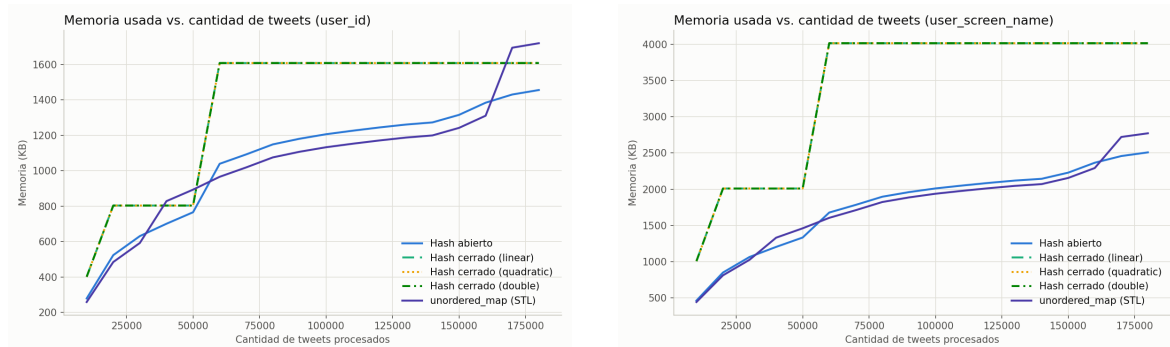


Figura 1: Memoria usada versus cantidad de tweets procesados. Izquierda: `user_id`. Derecha: `user_screen_name`. Las tres variantes de hashing cerrado se superponen exactamente (trazos entrelazados).

El hashing abierto es el más económico: sus listas enlazadas crecen exactamente con las claves insertadas, mientras que el hashing cerrado paga las celdas vacías que le impone su factor de carga máximo de 0,5 — penalización especialmente visible con claves string, donde cada celda vacía ocupa el tamaño completo de un `std::string` (3,92 MB frente a 2,45 MB del abierto). Los “escalones” de las curvas corresponden a los rehash: más pronunciados en el hashing cerrado, que debe mantener al menos el doble de celdas que de claves.

Rendimiento en la Creación y Escalabilidad

La siguiente tabla muestra el tiempo de conteo del dataset completo (media $\pm$ desviación estándar sobre las 20 repeticiones, en milisegundos):

Estructura	Con user_id (ms)	Con user_screen_name (ms)
Hashing abierto	13,0 $\pm$ 1,9	22,2 $\pm$ 8,9
Hashing cerrado (linear)	8,6 $\pm$ 2,6	16,0 $\pm$ 1,2
Hashing cerrado (quadratic)	8,3 $\pm$ 1,2	17,0 $\pm$ 5,9
Hashing cerrado (double)	10,9 $\pm$ 2,1	20,0 $\pm$ 2,2
<code>std::unordered_map</code>	11,0 $\pm$ 1,3	20,0 $\pm$ 7,1

Todas las estructuras escalan de forma aproximadamente lineal con el volumen de datos, consistente con un costo esperado $O(1)$ por operación. El hashing cerrado con sondeo cuadrático y lineal fueron las variantes más rápidas con ambas claves, superando incluso a `std::unordered_map`: sus celdas contiguas en un arreglo aprovechan mejor la caché del procesador que los nodos enlazados dispersos en el heap del hashing abierto,

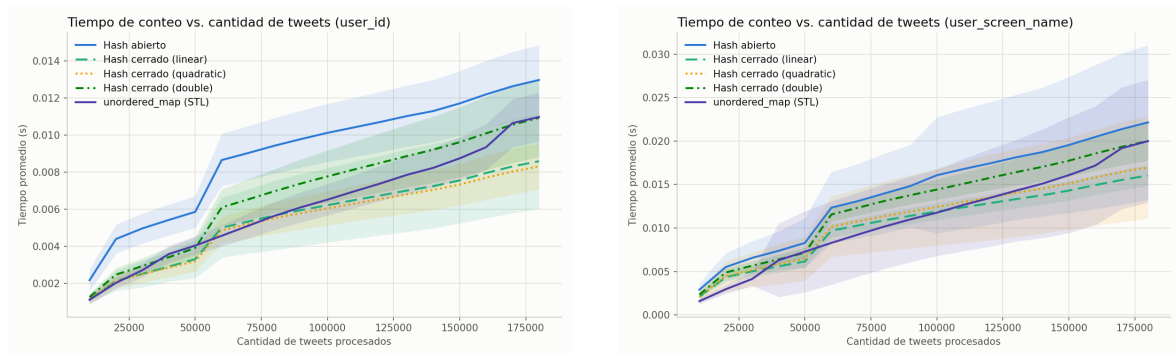


Figura 2: Tiempo promedio de conteo versus cantidad de tweets procesados (20 repeticiones; banda sombreada: ± 1 desviación estándar). Izquierda: `user_id`. Derecha: `user_screen_name`.

que resultó el más lento. El double hashing paga el costo de evaluar una segunda función hash en cada colisión. Los saltos visibles cerca de los 20.000 y 60.000 tweets corresponden a los rehash.

Análisis del Impacto del Tipo de Clave

El contraste entre claves es consistente en las cinco configuraciones: contar con `user_id` es entre un 45 % y un 95 % más rápido que con `user_screen_name`. Con el dataset completo, quadratic probing tarda 8,3 ms con la clave numérica frente a 17,0 ms con la textual, y el patrón se repite en el resto (hashing abierto: 13,0 vs. 22,2 ms; `unordered_map`: 11,0 vs. 20,0 ms).

La explicación es doble. Primero, **hashear un entero es más barato**: SplitMix64 ejecuta una cantidad fija y pequeña de operaciones aritméticas, mientras que la acumulación polinomial debe recorrer la cadena carácter por carácter. Segundo, **comparar enteros es más barato**: cada verificación de igualdad durante el sondeo de colisiones cuesta una instrucción con `uint64_t`, pero un recorrido byte a byte con `std::string`. El efecto se replica en memoria: las claves string aumentan el consumo entre un 61 % y un 150 % según la estructura, porque cada clave almacenada (y cada celda vacía, en el hashing cerrado) carga con el tamaño del objeto string completo.

Conclusiones

Este trabajo comparó experimentalmente tres tablas hash — hashing abierto propio, hashing cerrado propio con tres estrategias de sondeo, y `std::unordered_map` — contando los tweets de 45.862 usuarios sobre un dataset real de 183.370 registros, con dos tipos de clave.

El hashing cerrado fue el más rápido. Sus variantes quadratic y linear probing lideraron con ambas claves (8,3 y 8,6 ms con `user_id`), superando incluso a la biblioteca estándar (11,0 ms). Una implementación propia con buena localidad de caché — celdas contiguas en un arreglo — y un factor de carga bajo puede competir con la STL en una carga de trabajo específica. El hashing abierto fue consistentemente el más lento (13,0 ms), penalizado por la dispersión de sus nodos en el heap.

La clave importa tanto como la estructura. `user_id` (entero de 64 bits) resultó entre un 45 % y un 95 % más rápido que `user_screen_name` (string) en todas las configuraciones, y entre un 61 % y un 150 % más económico en memoria, por el menor costo tanto del cálculo del hash como de las comparaciones de igualdad durante el sondeo. Cuando existen ambas opciones como identificador único, la clave numérica es preferible sin ambigüedad.

Existe un compromiso claro entre tiempo y memoria. El hashing cerrado compra su velocidad reservando al menos el doble de celdas que de claves (factor de carga máximo 0,5), mientras que el hashing abierto minimiza la memoria creciendo solo con las claves efectivamente insertadas (1,42 MB vs. 1,57 MB con `user_id`, y 2,45 MB vs. 3,92 MB con string, donde cada celda vacía del cerrado carga con un `std::string` completo). Adicionalmente, se verificó que las tres estrategias de sondeo consumen memoria idéntica — difieren únicamente en tiempo —, por lo que la elección entre ellas debe basarse solo en ese criterio.

Para la tarea evaluada, la configuración recomendada es **hashing cerrado con quadratic probing y clave `user_id`**: la combinación más rápida, con un sobre costo de memoria moderado frente al hashing abierto y sin la penalización de la segunda función hash del double hashing.

Referencias

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [2] M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Pearson, 2014.
- [3] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [4] G. L. Steele Jr., D. Lea, C. H. Flood, “Fast Splittable Pseudorandom Number Generators,” en *Proc. ACM SIGPLAN Int. Conf. on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, ACM, pp. 453–472, 2014.